

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«СЕВЕРО–КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии
Департамент цифровых, робототехнических систем и электроники

ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №8
дисциплины
«Основы нейронных сетей»
Вариант 10

Выполнил:
Рябинин Егор Алексеевич
3 курс, группа ИВТ-б-о-23-2,
09.03.01 «Информатика и
вычислительная техника»,
направленность (профиль)
«Программное обеспечение средств
вычислительной техники и
автоматизированных систем», очная
форма обучения

(подпись)

Проверил:
Доцент департамента цифровых,
робототехнических систем и
электроники института перспективной
инженерии
Воронкин Роман Александрович

(подпись)

Отчет защищен с оценкой _____ Дата защиты _____

Ставрополь, 2026 г

Тема: анализ и классификация аудиосигналов с применением нейронных сетей

Цель: изучить методологию анализа, предобработки и классификации аудиоданных с использованием аппарата глубоких нейронных сетей. Реализовать, оптимизировать и провести сравнительный анализ архитектур (Dense, Conv1D) и подходов к извлечению признаков (MFCC, хромограммы, спектральные характеристики) для задач классификации музыкальных жанров и распознавания эмоций в речи. Закрепить навыки обработки аудиосигналов, работы с мел-спектрограммами и кросс-датасетной валидации.

Порядок выполнения работы:

Ссылка на репозиторий:

https://github.com/bohemiaaaaa/Lab8_Neural-networks

Pytorch:

Задание 1. Запустите, пожалуйста, раздел "Подготовка" и приступайте к выполнению заданий.

```
# Массивы и работа с данными
import numpy as np
import pandas as pd

# Отрисовка графиков
import matplotlib.pyplot as plt

# Загрузка из google облака
import gdown

# Работа с папками и файлами
import os
import time
import random
import math
import pickle

# PyTorch
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import DataLoader, TensorDataset, random_split

# Параметризация аудио
import librosa

# Разбиение на обучающую и проверочную выборку
from sklearn.model_selection import train_test_split

# Кодирование категориальных меток, нормирование числовых данных
from sklearn.preprocessing import LabelEncoder, StandardScaler

# Матрица ошибок классификатора
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay

# Отключение предупреждений
import warnings
warnings.filterwarnings('ignore')

# Для воспроизводимости
torch.manual_seed(42)
np.random.seed(42)
random.seed(42)
```

Рисунок 1 – Импорт библиотек

```
# Загрузка датасета из облака
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/112/genres.zip', None, quiet=False)

# Загрузка подготовленных данных
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/112/audio_data_mean.pickle', None, quiet=True)
```

Рисунок 2 – Загрузка датасета и подготовка данных

```
# Установка констант

FILE_DIR = './genres' # Папка с файлами датасета
CLASS_LIST = os.listdir(FILE_DIR) # Список классов, порядок методов
CLASS_LIST.sort() # Сортировка списка классов для воспроизводимости
CLASS_COUNT = len(CLASS_LIST) # Количество классов
CLASS_FILES = 100 # Общее количество файлов в каждом классе
FILE_INDEX_TRAIN_SPLIT = 90 # Количество файлов каждого класса в обучающей выборке
VALIDATION_SPLIT = 0.1 # Доля проверочной выборки в обучающей выборке
DURATION_SEC = 30 # Анализируемая длительность аудио
N_FFT = 8192 # Размер окна преобразования Фурье
HOP_LENGTH = 512 # Объем данных для расчета одного спектрограммного кадра

# PyTorch устройство
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
print(f"Используется устройство: {device}")
```

Рисунок 3 – Установка констант

```

# Функция параметризации аудио

def get_features(y, sr, n_fft=N_FFT, hop_length=HOP_LENGTH):
    # волновое представление сигнала
    # частота дискретизации сигнала y
    # размер скользящего окна БПФ
    # шаг скользящего окна БПФ

    # Вычисление различных параметров (признаков) аудио

    # Хромограмма
    chroma_stft = librosa.feature.chroma_stft(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # Мел-кепстральные коэффициенты
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # Среднеквадратическая амплитуда
    rmse = librosa.feature.rms(y=y, hop_length=hop_length)
    # Спектральный центроид
    spec_cent = librosa.feature.spectral_centroid(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # Ширина полосы частот
    spec_bw = librosa.feature.spectral_bandwidth(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # Спектральный спад частоты
    rolloff = librosa.feature.spectral_rolloff(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # Пересечения нуля
    zcr = librosa.feature.zero_crossing_rate(y, hop_length=hop_length)

    # Сборка параметров в общий список:
    # На один файл один усредненный вектор признаков
    features = {'rmse': rmse.mean(axis=1, keepdims=True),
               'spct': spec_cent.mean(axis=1, keepdims=True),
               'spbw': spec_bw.mean(axis=1, keepdims=True),
               'roff': rolloff.mean(axis=1, keepdims=True),
               'zcr': zcr.mean(axis=1, keepdims=True),
               'mfcc': mfcc.mean(axis=1, keepdims=True),
               'stft': chroma_stft.mean(axis=1, keepdims=True)}

    return features

```

Рисунок 4 – Функция параметризации аудио

```

def stack_features(feats):
    features = None
    for v in feats.values():
        features = np.vstack((features, v)) if features is not None else v
    return features.T

```

Рисунок 5 – Функция объединения признаков в набор векторов

```
def get_feature_list_from_file(class_index, song_name, duration_sec):
    y, sr = librosa.load(song_name, mono=True, duration=duration_sec)
    features = get_features(y, sr)
    feature_set = stack_features(features)

    # Возвращаем индекс класса (не one-hot, преобразуем позже)
    y_label = class_index

    return feature_set, y_label
```

Рисунок 6 – Функция формирования набора признаков и метки класса для аудиофайла

```
def process_file(class_index, file_index, duration_sec):
    class_name = CLASS_LIST[class_index]
    song_name = f'{FILE_DIR}/{class_name}/{class_name}.{str(file_index).zfill(5)}.au'

    feature_set, y_label = get_feature_list_from_file(class_index, song_name, duration_sec)

    # Усредняем по времени (один вектор на файл)
    feature_mean = np.mean(feature_set, axis=0, keepdims=True)

    return song_name, feature_mean.astype('float32'), y_label
```

Рисунок 7 – Функция формирования подвыборки признаков и меток класса для одного файла

```
def extract_data(file_index_start, file_index_end, duration_sec=DURATION_SEC):
    x_data = None
    y_data = None

    curr_time = time.time()

    for class_index in range(len(CLASS_LIST)):
        for file_index in range(file_index_start, file_index_end):
            _, file_x_data, file_y_data = process_file(class_index, file_index, duration_sec)
            x_data = file_x_data if x_data is None else np.vstack([x_data, file_x_data])
            y_data = np.append(y_data, file_y_data) if y_data is not None else np.array([file_y_data])

        print(f'Жанр {CLASS_LIST[class_index]} готов -> {round(time.time() - curr_time)} c')
        curr_time = time.time()

    return x_data, y_data
```

Рисунок 8 – Функция формирования набора данных из файлов всех классов по диапазону номеров файлов

```

x_train, x_val, y_train, y_val = train_test_split(x_train_data_scaled,
                                                y_train_data,
                                                stratify=y_train_data,
                                                test_size=VALIDATION_SPLIT)

print(f"x_train: {x_train.shape}, y_train: {y_train.shape}")
print(f"x_val: {x_val.shape}, y_val: {y_val.shape}")

# Преобразование в тензоры PyTorch
x_train_tensor = torch.FloatTensor(x_train)
y_train_tensor = torch.LongTensor(y_train) # LongTensor для меток (int64)
x_val_tensor = torch.FloatTensor(x_val)
y_val_tensor = torch.LongTensor(y_val)      # LongTensor для меток

# Создание DataLoader
train_dataset = TensorDataset(x_train_tensor, y_train_tensor)
val_dataset = TensorDataset(x_val_tensor, y_val_tensor)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)

```

Рисунок 9 – Разделение набора данных на обучающую и проверочную выборки

```

def show_history(train_losses, val_losses, train_accs, val_accs):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 5))
    fig.suptitle('График процесса обучения модели')

    ax1.plot(train_accs, label='Доля верных ответов на обучающем наборе')
    ax1.plot(val_accs, label='Доля верных ответов на проверочном наборе')
    ax1.set_xlabel('Эпоха обучения')
    ax1.set_ylabel('Доля верных ответов')
    ax1.legend()

    ax2.plot(train_losses, label='Ошибка на обучающем наборе')
    ax2.plot(val_losses, label='Ошибка на проверочном наборе')
    ax2.set_xlabel('Эпоха обучения')
    ax2.set_ylabel('Ошибка')
    ax2.legend()
    plt.show()

```

Рисунок 10 – Вывод графиков точности и ошибки распознавания на обучающей и проверочной выборках

```

def classify_file(model, x_scaler, class_index, file_index):
    model.eval()

    song_name, file_x_data, file_y_data = process_file(class_index, file_index, DURATION_SEC)
    file_x_data = x_scaler.transform(file_x_data)

    print('Файл:', song_name)
    print('Вектор для предсказания:', file_x_data.shape)

    # Преобразование в тензор
    file_tensor = torch.FloatTensor(file_x_data).to(device)

    with torch.no_grad():
        predict = model(file_tensor)
        predict_mean = predict.cpu().numpy()[0]

    predict_class_index = np.argmax(predict_mean)
    predict_good = predict_class_index == class_index

    plt.figure(figsize=(10, 3))
    print('Классификация сети:', CLASS_LIST[predict_class_index], '-', 'ВЕРНО :-)' if predict_good else 'НЕВЕРНО.')
    plt.title('Среднее распределение векторов предсказаний')
    plt.bar(CLASS_LIST, predict_mean, color='g' if predict_good else 'r')
    plt.xticks(rotation=45)
    plt.show()
    print('-----')

    return predict_class_index

```

Рисунок 11 – Классификация файла и визуализация предсказания модели для него

```

def classify_test_files(model, x_scaler, from_index, n_files):
    model.eval()
    predict_all = 0
    predict_good = 0
    y_true = []
    y_pred = []

    for class_index in range(CLASS_COUNT):
        for file_index in range(from_index, from_index + n_files):
            predict_class_index = classify_file(model, x_scaler, class_index, file_index)
            y_true.append(class_index)
            y_pred.append(predict_class_index)
            predict_all += 1
            predict_good += (predict_class_index == class_index)

    good_ratio = round(predict_good / predict_all * 100., 2)
    print(f'=== Обработано образцов: {predict_all}, из них распознано верно: {predict_good}, доля верных: {good_ratio}% ===')

    cm = confusion_matrix(y_true, y_pred)
    fig, ax = plt.subplots(figsize=(10, 10))
    ax.set_title('Матрица ошибок по файлам аудио (не нормализованная)')
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=CLASS_LIST)
    disp.plot(ax=ax)
    plt.xticks(rotation=45)
    plt.show()

```

Рисунок 12 – Классификация и визуализация нескольких файлов каждого класса

1. Проверьте форму данных обучающей и проверочной выборок (выведите на экран).

```
print(f"Форма обучающей выборки x_train: {x_train.shape}")
print(f"Форма обучающей выборки y_train: {y_train.shape}")
print(f"Форма проверочной выборки x_val: {x_val.shape}")
print(f"Форма проверочной выборки y_val: {y_val.shape}")
print(f"Количество классов: {CLASS_COUNT}")
print(f"Жанры: {CLASS_LIST}")
```

Форма обучающей выборки x_train: (810, 37)
Форма обучающей выборки y_train: (810,)
Форма проверочной выборки x_val: (90, 37)
Форма проверочной выборки y_val: (90,)
Количество классов: 10
Жанры: ['blues', 'classical', 'country', 'disco', 'hiphop', 'jazz', 'metal', 'pop', 'reggae', 'rock']

Рисунок 13 – Форма данных обучающей и проверочной выборок

2. Составьте модель классификатора на полносвязных слоях и сохраните ее в переменной model. Для этого:

- Используйте заготовку для последовательной модели Sequential;
- Добавьте полносвязный слой на 64 нейрона с активационной функцией relu, после него добавьте слой Dropout с долей отключаемых нейронов 30%;
- Добавьте следующий полносвязный слой на 32 нейрона с активационной функцией relu, после него добавьте слой Dropout с долей отключаемых нейронов 30%;
- Добавьте следующий полносвязный слой на 16 нейронов с активационной функцией relu, после него добавьте слой Dropout с долей отключаемых нейронов 20%;
- Добавьте слой пакетной нормализации;
- Добавьте финальный полносвязный слой классификатора на число нейронов по числу классов (CLASS_COUNT) с активационной функцией softmax .

```

class AudioClassifier(nn.Module):
    def __init__(self, input_size, num_classes):
        super(AudioClassifier, self).__init__()
        self.fc1 = nn.Linear(input_size, 64)
        self.fc2 = nn.Linear(64, 32)
        self.fc3 = nn.Linear(32, 16)
        self.fc4 = nn.Linear(16, num_classes)
        self.dropout1 = nn.Dropout(0.3)
        self.dropout2 = nn.Dropout(0.3)
        self.dropout3 = nn.Dropout(0.2)
        self.batchnorm = nn.BatchNorm1d(16)

    def forward(self, x):
        x = torch.relu(self.fc1(x))
        x = self.dropout1(x)
        x = torch.relu(self.fc2(x))
        x = self.dropout2(x)
        x = torch.relu(self.fc3(x))
        x = self.dropout3(x)
        x = self.batchnorm(x)
        x = self.fc4(x)
        x = torch.softmax(x, dim=1)
        return x

model = AudioClassifier(x_train.shape[1], CLASS_COUNT).to(device)

```

Рисунок 14 – Создание модели

3. Откомпилируйте созданную модель методом `.compile()` с указанием оптимизатора Adam и начальным шагом обучения 0.0001, функцией ошибки `categorical_crossentropy` и метрикой `acc`; Выведите на экран сводку архитектуры полученной модели методом `.summary()`.

```

optimizer = torch.optim.Adam(model.parameters(), lr=0.0001)
criterion = nn.CrossEntropyLoss()

print(model)

AudioClassifier(
  (fc1): Linear(in_features=37, out_features=64, bias=True)
  (fc2): Linear(in_features=64, out_features=32, bias=True)
  (fc3): Linear(in_features=32, out_features=16, bias=True)
  (fc4): Linear(in_features=16, out_features=10, bias=True)
  (dropout1): Dropout(p=0.3, inplace=False)
  (dropout2): Dropout(p=0.3, inplace=False)
  (dropout3): Dropout(p=0.2, inplace=False)
  (batchnorm): BatchNorm1d(16, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
)

```

Рисунок 15 – Компиляция модели

4. Обучите модель и выведите графики обучения:

Обучите созданную и откомпилированную модель классификатора на данных обучающей выборки `x_train`, `y_train`, используя проверочные данные `x_val`, `y_val`, размер батча 32 и количество эпох 1000. Результаты обучения сохраните в переменной `history`.

В разделе "Функция вывода графиков точности и ошибки по эпохам обучения" найдите определение функции `show_history()`, изучите требуемые параметры для нее и используйте для построения графиков точности и ошибки на протяжении эпох обучения.

```
epochs = 1000
batch_size = 32

# DataLoader
train_loader = DataLoader(TensorDataset(x_train_tensor, y_train_tensor), batch_size=batch_size, shuffle=True)
val_loader = DataLoader(TensorDataset(x_val_tensor, y_val_tensor), batch_size=batch_size, shuffle=False)

train_losses, val_losses, train_accs, val_accs = [], [], [], []

for epoch in range(epochs):
    # Обучение
    model.train()
    train_loss, correct_train, total_train = 0, 0, 0
    for batch_x, batch_y in train_loader:
        batch_x, batch_y = batch_x.to(device), batch_y.to(device)
        optimizer.zero_grad()
        outputs = model(batch_x)
        loss = criterion(outputs, batch_y)
        loss.backward()
        optimizer.step()

        train_loss += loss.item()
        _, predicted = torch.max(outputs, 1)
        total_train += batch_y.size(0)
        correct_train += (predicted == batch_y).sum().item()

    # Валидация
    model.eval()
    val_loss, correct_val, total_val = 0, 0, 0
    with torch.no_grad():
        for batch_x, batch_y in val_loader:
            batch_x, batch_y = batch_x.to(device), batch_y.to(device)
            outputs = model(batch_x)
            loss = criterion(outputs, batch_y)

            val_loss += loss.item()
            _, predicted = torch.max(outputs, 1)
            total_val += batch_y.size(0)
            correct_val += (predicted == batch_y).sum().item()

    train_losses.append(train_loss / len(train_loader))
    val_losses.append(val_loss / len(val_loader))
    train_accs.append(correct_train / total_train)
    val_accs.append(correct_val / total_val)
```

Рисунок 16 – Обучение модели

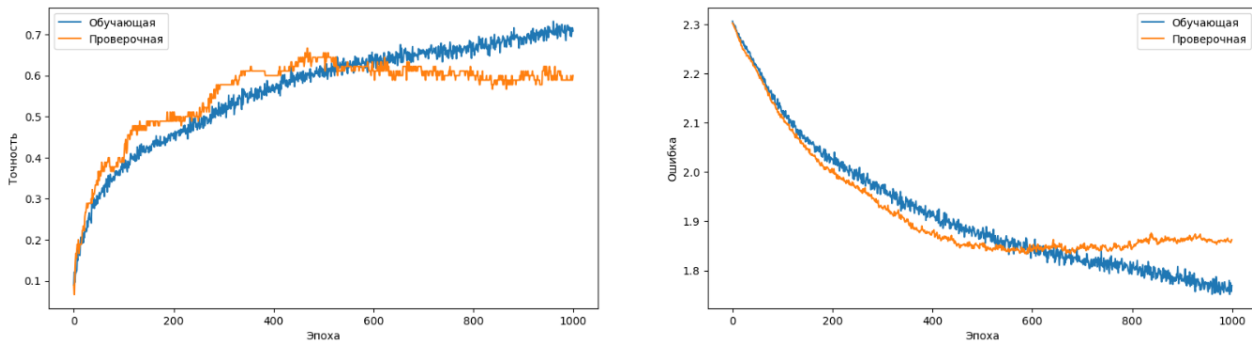


Рисунок 17 – Графики обучения

5. Проверьте работу модели:

В разделе Функции визуализации распознавания отдельных звуковых файлов найдите определение функции `classify_test_files()` и изучите ее параметры. Используйте функцию для визуализации работы классификатора на произвольном количестве тестовых звуковых файлов, полагая, что:

вы используете обученная в задании 4 модель классификатора аудио;

нормализатор `x_scaler` уже настроен ранее в ноутбуке, и его нужно передать в функцию `classify_test_files()` вместо параметра `x_scaler`;

тестовые звуковые файлы начинаются с индекса 90 и всего их ровно 10 для каждого класса.

```
classify_test_files(model=model,
                    x_scaler=x_scaler,
                    from_index=90,
                    n_files=10)
```

Рисунок 18 – Проверка работы модели

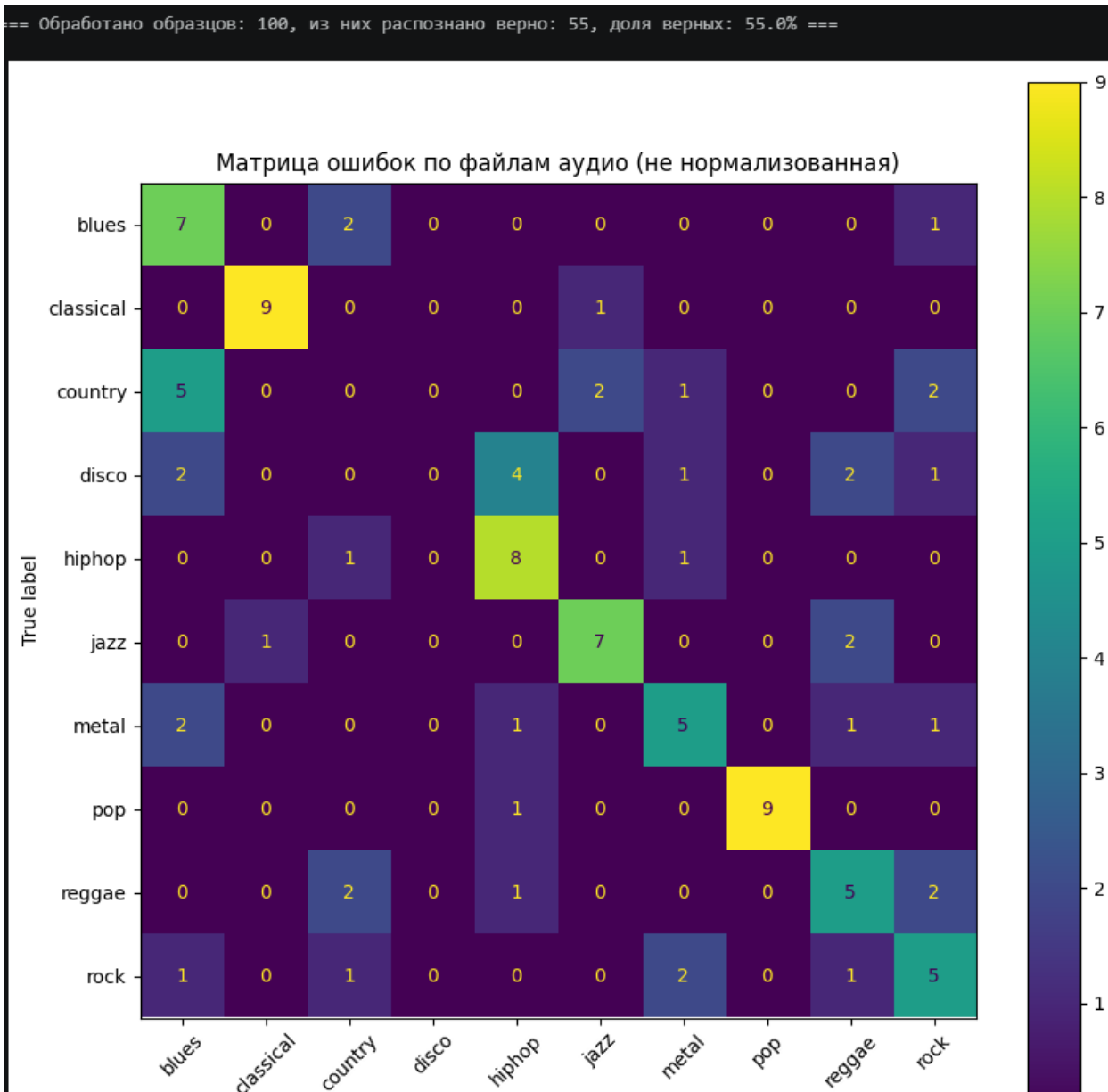


Рисунок 19 – Результат проверки и матрица ошибок

Tensorflow:

Задание 1. Запустите, пожалуйста, раздел "Подготовка" и приступайте к выполнению заданий.

```

# Массивы
import numpy as np

# Отрисовка графиков
import matplotlib.pyplot as plt

# Загрузка из google облака
import gdown

# Преобразование категориальных данных в one hot encoding
from tensorflow.keras.utils import to_categorical

# Работа с папками и файлами
import os

# Утилиты работы со временем
import time

# Работа со случайными числами
import random

# Математические функции
import math

# Сохранение и загрузка структур данных Python
import pickle

# Параметризация аудио
import librosa

# Оптимизаторы для обучения моделей
from tensorflow.keras.optimizers import Adam, RMSprop

# Конструирование и загрузка моделей нейронных сетей
from tensorflow.keras.models import Sequential, Model, load_model

# Основные слои
from tensorflow.keras.layers import concatenate, Input, Dense, Dropout, BatchNormalization, Flatten, Conv1D, Conv2D, LSTM

# Разбиение на обучающую и проверочную выборку
from sklearn.model_selection import train_test_split

# Кодирование категориальных меток, нормирование числовых данных
from sklearn.preprocessing import LabelEncoder, StandardScaler

```

Рисунок 20 – Импорт библиотек

```

# Загрузка датасета из облака
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l12/genres.zip', None, quiet=False)

# Загрузка подготовленных данных
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l12/audio_data_mean.pickle', None, quiet=True)

```

Рисунок 21 – Загрузка датасета и подготовка данных

```

# Установка констант

FILE_DIR = './genres'
CLASS_LIST = os.listdir(FILE_DIR)
CLASS_LIST.sort()
CLASS_COUNT = len(CLASS_LIST)
CLASS_FILES = 100
FILE_INDEX_TRAIN_SPLIT = 90
VALIDATION_SPLIT = 0.1
DURATION_SEC = 30
N_FFT = 8192
HOP_LENGTH = 512

# Папка с файлами датасета
# Список классов, порядок мет
# Сортировка списка классов д
# Количество классов
# Общее количество файлов в к
# Количество файлов каждого к
# Доля проверочной выборки в
# Анализируемая длительность
# Размер окна преобразования
# Объем данных для расчета од

```

Рисунок 22 – Установка констант

```

# Функция параметризации аудио

def get_features(y, sr, n_fft=N_FFT, hop_length=HOP_LENGTH):
    # волновое представление сигнала
    # частота дискретизации сигнала y
    # размер скользящего окна БПФ
    # шаг скользящего окна БПФ

    # Вычисление различных параметров (признаков) аудио

    # Хромограмма
    chroma_stft = librosa.feature.chroma_stft(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # Мел-кепстральные коэффициенты
    mfcc = librosa.feature.mfcc(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # Среднеквадратическая амплитуда
    rmse = librosa.feature.rms(y=y, hop_length=hop_length)
    # Спектральный центроид
    spec_cent = librosa.feature.spectral_centroid(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # Ширина полосы частот
    spec_bw = librosa.feature.spectral_bandwidth(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # Спектральный спад частоты
    rolloff = librosa.feature.spectral_rolloff(y=y, sr=sr, n_fft=n_fft, hop_length=hop_length)
    # Пересечения нуля
    zcr = librosa.feature.zero_crossing_rate(y, hop_length=hop_length)

    # Сборка параметров в общий список:
    # На один файл один усредненный вектор признаков
    features = {'rmse': rmse.mean(axis=1, keepdims=True),
               'spct': spec_cent.mean(axis=1, keepdims=True),
               'spbw': spec_bw.mean(axis=1, keepdims=True),
               'roff': rolloff.mean(axis=1, keepdims=True),
               'zcr': zcr.mean(axis=1, keepdims=True),
               'mfcc': mfcc.mean(axis=1, keepdims=True),
               'stft': chroma_stft.mean(axis=1, keepdims=True)}

    return features

```

Рисунок 23 – Функция параметризации аудио

```

def stack_features(feats):
    features = None
    for v in feats.values():
        features = np.vstack((features, v)) if features is not None else v
    return features.T

```

Рисунок 24 – Функция объединения признаков в набор векторов

```
def get_feature_list_from_file(class_index, song_name, duration_sec):
    y, sr = librosa.load(song_name, mono=True, duration=duration_sec)
    features = get_features(y, sr)
    feature_set = stack_features(features)

    # Возвращаем индекс класса (не one-hot, преобразуем позже)
    y_label = class_index

    return feature_set, y_label
```

Рисунок 25 – Функция формирования набора признаков и метки класса для аудиофайла

```
def process_file(class_index, file_index, duration_sec):
    class_name = CLASS_LIST[class_index]
    song_name = f'{FILE_DIR}/{class_name}/{class_name}.{str(file_index).zfill(5)}.au'

    feature_set, y_label = get_feature_list_from_file(class_index, song_name, duration_sec)

    # Усредняем по времени (один вектор на файл)
    feature_mean = np.mean(feature_set, axis=0, keepdims=True)

    return song_name, feature_mean.astype('float32'), y_label
```

Рисунок 26 – Функция формирования подвыборки признаков и меток класса для одного файла

```
def extract_data(file_index_start, file_index_end, duration_sec=DURATION_SEC):
    x_data = None
    y_data = None

    curr_time = time.time()

    for class_index in range(len(CLASS_LIST)):
        for file_index in range(file_index_start, file_index_end):
            _, file_x_data, file_y_data = process_file(class_index, file_index, duration_sec)
            x_data = file_x_data if x_data is None else np.vstack([x_data, file_x_data])
            y_data = np.append(y_data, file_y_data) if y_data is not None else np.array([file_y_data])

        print(f'Жанр {CLASS_LIST[class_index]} готов -> {round(time.time() - curr_time)} c')
        curr_time = time.time()

    return x_data, y_data
```

Рисунок 27 – Функция формирования набора данных из файлов всех классов по диапазону номеров файлов


```

x_train, x_val, y_train, y_val = train_test_split(x_train_data_scaled,
                                                y_train_data,
                                                stratify=y_train_data,
                                                test_size=VALIDATION_SPLIT)

print(f"x_train: {x_train.shape}, y_train: {y_train.shape}")
print(f"x_val: {x_val.shape}, y_val: {y_val.shape}")

# Преобразование в тензоры PyTorch
x_train_tensor = torch.FloatTensor(x_train)
y_train_tensor = torch.LongTensor(y_train) # LongTensor для меток (int64)
x_val_tensor = torch.FloatTensor(x_val)
y_val_tensor = torch.LongTensor(y_val)      # LongTensor для меток

# Создание DataLoader
train_dataset = TensorDataset(x_train_tensor, y_train_tensor)
val_dataset = TensorDataset(x_val_tensor, y_val_tensor)

train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=32, shuffle=False)

```

Рисунок 28 – Разделение набора данных на обучающую и проверочную выборки

```

def show_history(train_losses, val_losses, train_accs, val_accs):
    fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20, 5))
    fig.suptitle('График процесса обучения модели')

    ax1.plot(train_accs, label='Доля верных ответов на обучающем наборе')
    ax1.plot(val_accs, label='Доля верных ответов на проверочном наборе')
    ax1.set_xlabel('Эпоха обучения')
    ax1.set_ylabel('Доля верных ответов')
    ax1.legend()

    ax2.plot(train_losses, label='Ошибка на обучающем наборе')
    ax2.plot(val_losses, label='Ошибка на проверочном наборе')
    ax2.set_xlabel('Эпоха обучения')
    ax2.set_ylabel('Ошибка')
    ax2.legend()
    plt.show()

```

Рисунок 29 – Вывод графиков точности и ошибки распознавания на обучающей и проверочной выборках

```
def classify_file(model, x_scaler, class_index, file_index):
    model.eval()

    song_name, file_x_data, file_y_data = process_file(class_index, file_index, DURATION_SEC)
    file_x_data = x_scaler.transform(file_x_data)

    print('Файл:', song_name)
    print('Вектор для предсказания:', file_x_data.shape)

    # Преобразование в тензор
    file_tensor = torch.FloatTensor(file_x_data).to(device)

    with torch.no_grad():
        predict = model(file_tensor)
        predict_mean = predict.cpu().numpy()[0]

    predict_class_index = np.argmax(predict_mean)
    predict_good = predict_class_index == class_index

    plt.figure(figsize=(10, 3))
    print('Классификация сети:', CLASS_LIST[predict_class_index], '-', 'ВЕРНО :-)' if predict_good else 'НЕВЕРНО.')
    plt.title('Среднее распределение векторов предсказаний')
    plt.bar(CLASS_LIST, predict_mean, color='g' if predict_good else 'r')
    plt.xticks(rotation=45)
    plt.show()
    print('-----')

    return predict_class_index
```

Рисунок 30 – Классификация файла и визуализация предсказания модели для него

```
def classify_test_files(model, x_scaler, from_index, n_files):
    model.eval()
    predict_all = 0
    predict_good = 0
    y_true = []
    y_pred = []

    for class_index in range(CLASS_COUNT):
        for file_index in range(from_index, from_index + n_files):
            predict_class_index = classify_file(model, x_scaler, class_index, file_index)
            y_true.append(class_index)
            y_pred.append(predict_class_index)
            predict_all += 1
            predict_good += (predict_class_index == class_index)

    good_ratio = round(predict_good / predict_all * 100., 2)
    print(f'=== Обработано образцов: {predict_all}, из них распознано верно: {predict_good}, доля верных: {good_ratio}% ===')

    cm = confusion_matrix(y_true, y_pred)
    fig, ax = plt.subplots(figsize=(10, 10))
    ax.set_title('Матрица ошибок по файлам аудио (не нормализованная)')
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=CLASS_LIST)
    disp.plot(ax=ax)
    plt.xticks(rotation=45)
    plt.show()
```

Рисунок 31 – Классификация и визуализация нескольких файлов каждого класса

1. Проверьте форму данных обучающей и проверочной выборок (выведите на экран).

```
print(f"Форма обучающей выборки x_train: {x_train.shape}")
print(f"Форма обучающей выборки y_train: {y_train.shape}")
print(f"Форма проверочной выборки x_val: {x_val.shape}")
print(f"Форма проверочной выборки y_val: {y_val.shape}")
```

```
Форма обучающей выборки x_train: (810, 37)
Форма обучающей выборки y_train: (810, 10)
Форма проверочной выборки x_val: (90, 37)
Форма проверочной выборки y_val: (90, 10)
```

Рисунок 32 – Форма данных

2. Составьте модель классификатора на полносвязных слоях и сохраните ее в переменной model. Для этого:

- Используйте заготовку для последовательной модели Sequential;
- Добавьте полносвязный слой на 64 нейрона с активационной функцией relu, после него добавьте слой Dropout с долей отключаемых нейронов 30%;
- Добавьте следующий полносвязный слой на 32 нейрона с активационной функцией relu, после него добавьте слой Dropout с долей отключаемых нейронов 30%;
- Добавьте следующий полносвязный слой на 16 нейронов с активационной функцией relu, после него добавьте слой Dropout с долей отключаемых нейронов 20%;
- Добавьте слой пакетной нормализации;
- Добавьте финальный полносвязный слой классификатора на число нейронов по числу классов (CLASS_COUNT) с активационной функцией softmax .

```
model = Sequential()

model.add(Dense(64, activation='relu', input_shape=(x_train.shape[1],)))
model.add(Dropout(0.3))

model.add(Dense(32, activation='relu'))
model.add(Dropout(0.3))

model.add(Dense(16, activation='relu'))
model.add(Dropout(0.2))

model.add(BatchNormalization())

model.add(Dense(CLASS_COUNT, activation='softmax'))
```

Рисунок 33 – Создание модели

3. Откомпилируйте созданную модель методом `.compile()` с указанием оптимизатора Adam и начальным шагом обучения 0.0001, функцией ошибки `categorical_crossentropy` и метрикой `accuracy`; Выведите на экран сводку архитектуры полученной модели методом `.summary()`.

<pre> model.compile(optimizer=Adam(learning_rate=0.0001), loss='categorical_crossentropy', metrics=['accuracy']) model.summary() </pre>		
Model: "sequential_1"		
Layer (type)	Output Shape	Param #
dense_4 (Dense)	(None, 64)	2,432
dropout_3 (Dropout)	(None, 64)	0
dense_5 (Dense)	(None, 32)	2,080
dropout_4 (Dropout)	(None, 32)	0
dense_6 (Dense)	(None, 16)	528
dropout_5 (Dropout)	(None, 16)	0
batch_normalization_1 (BatchNormalization)	(None, 16)	64
dense_7 (Dense)	(None, 10)	170

Рисунок 34 – Компиляция модели

4. Обучите модель и выведите графики обучения:

Обучите созданную и откомпилированную модель классификатора на данных обучающей выборки `x_train`, `y_train`, используя проверочные данные `x_val`, `y_val`, размер батча 32 и количество эпох 1000. Результаты обучения сохраните в переменной `history`.

В разделе "Функция вывода графиков точности и ошибки по эпохам обучения" найдите определение функции `show_history()`, изучите требуемые параметры для нее и используйте для построения графиков точности и ошибки на протяжении эпох обучения.

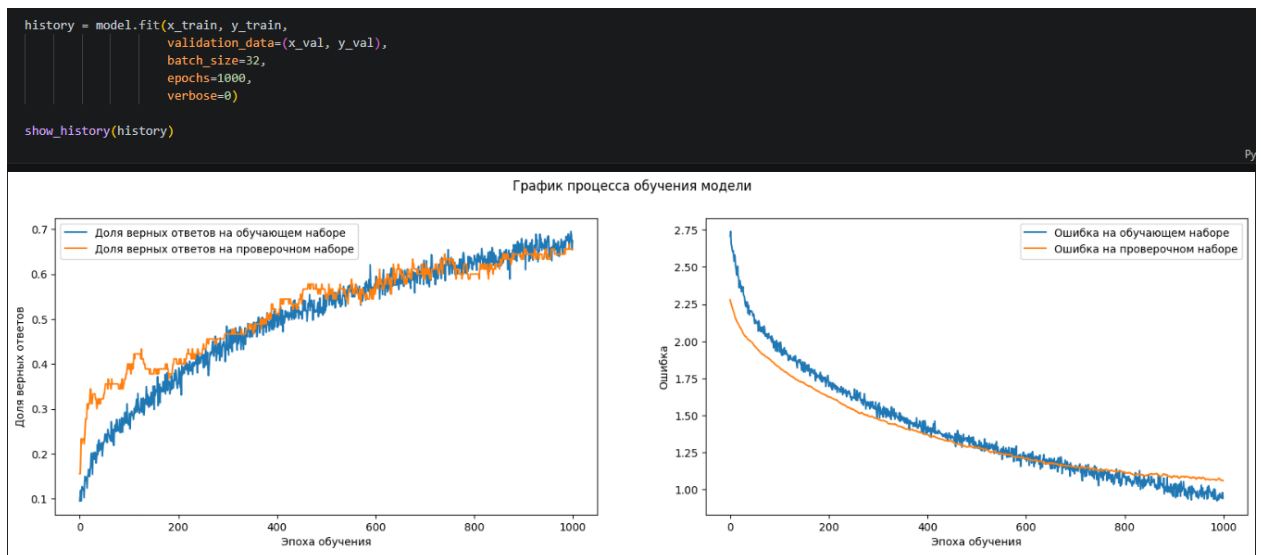


Рисунок 35 – Обучение модели и визуализация

5. Проверьте работу модели:

В разделе Функции визуализации распознавания отдельных звуковых файлов найдите определение функции `classify_test_files()` и изучите ее параметры. Используйте функцию для визуализации работы классификатора на произвольном количестве тестовых звуковых файлов, полагая, что:

вы используете обученная в задании 4 модель классификатора аудио;

нормализатор `x_scaler` уже настроен ранее в ноутбуке, и его нужно передать в функцию `classify_test_files()` вместо параметра `x_scaler`;

тестовые звуковые файлы начинаются с индекса 90 и всего их ровно 10 для каждого класса.

```
classify_test_files(model=model,
                    x_scaler=x_scaler,
                    from_index=90,
                    n_files=10)
```

Рисунок 36 – Проверка работы модели

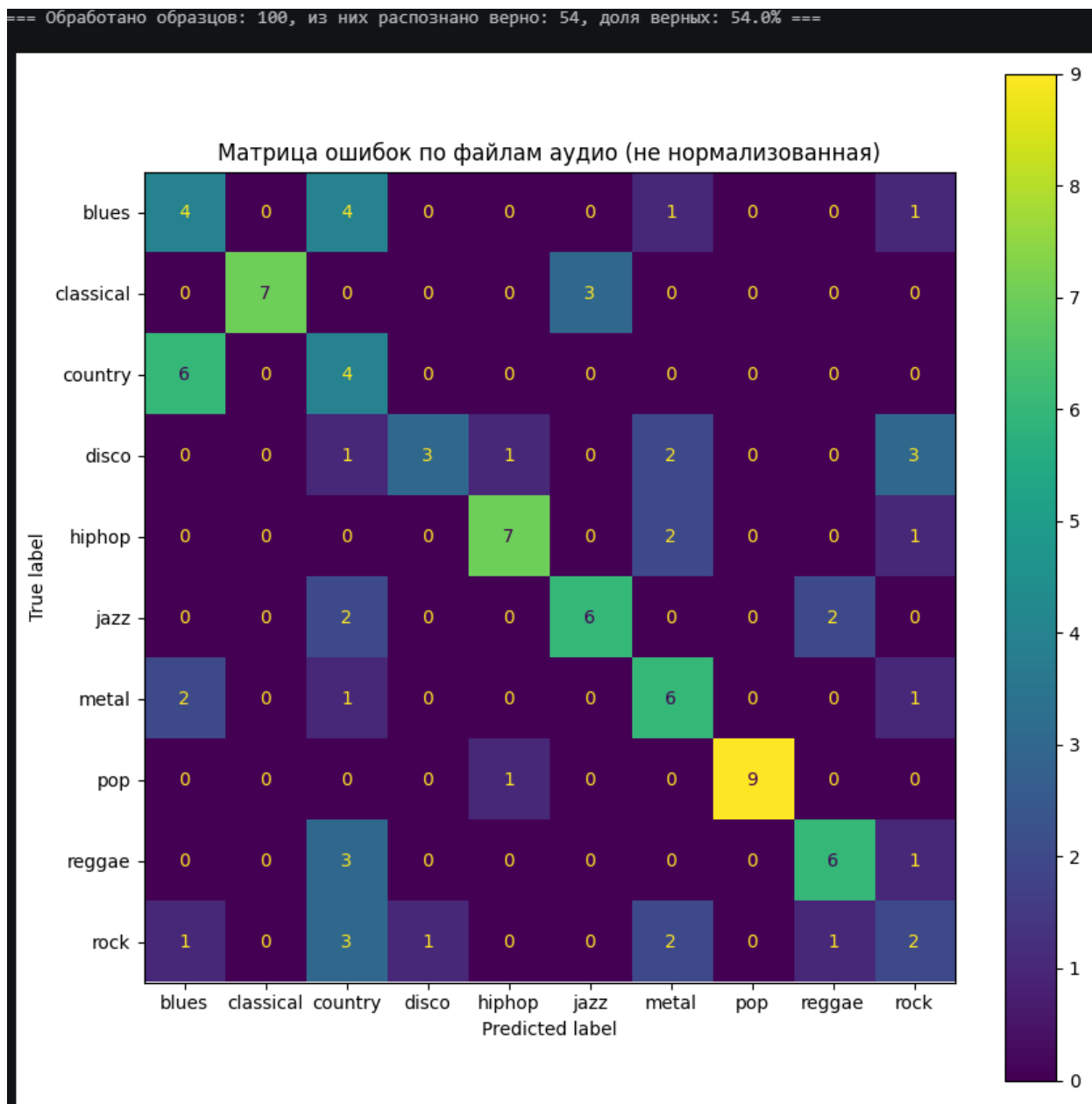


Рисунок 37 – Результат проверки и матрица ошибок

Задание 2. Используя базу "Аудиожанры", примените подход к музыке как к тексту и напишите сверточный классификатор (на базе слоя Conv1D) для подготовленных данных. Для этого:

1. Измените подготовку данных так, чтобы набор признаков, извлекаемый из аудиофайла, был представлен в виде последовательностей векторов признаков. Последовательности должны быть фиксированного размера и выбираться скользящим окном с заданным шагом. Другими словами: берем аудио-файл длительность, например, 30 сек. Берем отрезок фиксированной длины (например, 5с) и получаем набор признаков для этого

отрезка. Смещаемся на шаг (например, 1с) и берем следующий отрезок. Таким образом готовим обучающую выборку.

2. Длину последовательности, размер шага и достаточный набор признаков определите самостоятельно исходя из требований к точности классификатора;

3. Разработайте классификатор на одномерных сверточных слоях Conv1D с точностью классификации жанра на тестовых данных не ниже 60%, а на обучающих файлах - 68% и выше;

4. Используйте за основу материал с урока, но при желании разработайте свои инструменты.

```
# Массивы
import numpy as np

# Отрисовка графиков
import matplotlib.pyplot as plt

# Загрузка из google облака
import gdown

# Преобразование категориальных данных в one hot encoding
from tensorflow.keras.utils import to_categorical

# Работа с папками и файлами
import os

# Утилиты работы со временем
import time

# Работа со случайными числами
import random

# Математические функции
import math

# Сохранение и загрузка структур данных Python
import pickle

# Параметризация аудио
import librosa

# Оптимизаторы для обучения моделей
from tensorflow.keras.optimizers import Adam, RMSprop

# Конструирование и загрузка моделей нейронных сетей
from tensorflow.keras.models import Sequential, Model, load_model

# Основные слои
from tensorflow.keras.layers import concatenate, Input, Dense, Dropout, BatchNormalization, Flatten, Conv1D, Conv2D, LSTM, GlobalAveragePooling1D
from tensorflow.keras.layers import MaxPooling1D, AveragePooling1D, SpatialDropout1D

# Разбиение на обучающую и проверочную выборку
from sklearn.model_selection import train_test_split

# Кодирование категориальных меток, нормирование числовых данных
from sklearn.preprocessing import LabelEncoder, StandardScaler
```

Рисунок 38 – Импорт библиотек

```
# Загрузка датасета из облака
gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l12/genres.zip', None, quiet=True)

'genres.zip'
```

Рисунок 39 – Загрузка датасета


```

SR = 22050
DURATION_SEC = 30
WINDOW_SEC = 5
HOP_SEC = 2
N_FFT = 2048
HOP_LENGTH = 512
N_MFCC = 20

WINDOW_LEN = WINDOW_SEC * SR
HOP_LEN = HOP_SEC * SR
MAX_SEQ_LEN = (DURATION_SEC - WINDOW_SEC) // HOP_SEC + 1

```

Рисунок 40 – Параметры аудио

```

def extract_features(y, sr, start):
    end = start + WINDOW_LEN
    window = y[start:end]
    if len(window) < WINDOW_LEN:
        return None

    mfcc = librosa.feature.mfcc(y=window, sr=sr, n_mfcc=N_MFCC, n_fft=N_FFT, hop_length=HOP_LENGTH)
    rms = librosa.feature.rms(y=window, hop_length=HOP_LENGTH)
    cent = librosa.feature.spectral_centroid(y=window, sr=sr, n_fft=N_FFT, hop_length=HOP_LENGTH)

    return np.concatenate([np.mean(mfcc, axis=1), [np.mean(rms)], [np.mean(cent)]])

```

Рисунок 41 – Функция extract_features

```

def file_to_sequence(file_path):
    y, _ = librosa.load(file_path, sr=SR, duration=DURATION_SEC)
    seq = []
    for start in range(0, len(y) - WINDOW_LEN + 1, HOP_LEN):
        feat = extract_features(y, SR, start)
        if feat is not None:
            seq.append(feat)
        if len(seq) >= MAX_SEQ_LEN:
            break
    while len(seq) < MAX_SEQ_LEN:
        seq.append(np.zeros(N_MFCC + 2))
    return np.array(seq)

```

Рисунок 42 – Функция file_to_sequence (преобразование файла в последовательность векторов)

```

CLASS_LIST = sorted(os.listdir('./genres'))
CLASS_COUNT = len(CLASS_LIST)

X, Y = [], []
for class_idx, name in enumerate(CLASS_LIST):
    for f in range(90):
        path = f'./genres/{name}/{name}.{str(f).zfill(5)}.au'
        X.append(file_to_sequence(path))
        Y.append(class_idx)
    print(f'{name} готов')

X = np.array(X)
Y = to_categorical(Y, CLASS_COUNT)

print(f'X shape: {X.shape}, Y shape: {Y.shape}')

blues готов
classical готов
country готов
disco готов
hiphop готов
jazz готов
metal готов
pop готов
reggae готов
rock готов
X shape: (900, 13, 22), Y shape: (900, 10)

```

Рисунок 43 – Подготовка данных

```

scaler = StandardScaler()
shape = X.shape
X = scaler.fit_transform(X.reshape(-1, shape[-1])).reshape(shape)

```

Рисунок 44 – Нормирование данных

```

x_train, x_val, y_train, y_val = train_test_split(X, Y, stratify=Y, test_size=0.1)
print(f'x_train: {x_train.shape}, x_val: {x_val.shape}')

x train: (810, 13, 22), x val: (90, 13, 22)

```

Рисунок 45 – Разделение на обучающую и валидационную выборки

```

model = Sequential([
    Input(shape=(MAX_SEQ_LEN, N_MFCC + 2)),

    Conv1D(128, 5, activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling1D(2),
    Dropout(0.3),

    Conv1D(256, 5, activation='relu', padding='same'),
    BatchNormalization(),
    MaxPooling1D(2),
    Dropout(0.3),

    Conv1D(512, 3, activation='relu', padding='same'),
    BatchNormalization(),
    GlobalAveragePooling1D(),
    Dropout(0.4),

    Dense(256, activation='relu'),
    BatchNormalization(),
    Dropout(0.5),
    Dense(128, activation='relu'),
    Dropout(0.3),
    Dense(CLASS_COUNT, activation='softmax')
])

model.compile(optimizer=Adam(learning_rate=0.0005),
              loss='categorical_crossentropy',
              metrics=['accuracy'])

model.summary()

```

Рисунок 46 – Создание модели

```

from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau

early_stop = EarlyStopping(monitor='val_accuracy', patience=30, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=10, min_lr=1e-6)

history = model.fit(x_train, y_train,
                    validation_data=(x_val, y_val),
                    batch_size=16,
                    epochs=200,
                    callbacks=[early_stop, reduce_lr],
                    verbose=1)

# Графики
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(20,5))
ax1.plot(history.history['accuracy'], label='Train acc')
ax1.plot(history.history['val_accuracy'], label='Val acc')
ax1.legend()
ax2.plot(history.history['loss'], label='Train loss')
ax2.plot(history.history['val_loss'], label='Val loss')
ax2.legend()
plt.show()

print(f"Train accuracy: {history.history['accuracy'][-1]:.4f}")
print(f"Val accuracy: {history.history['val_accuracy'][-1]:.4f}")

```

Рисунок 47 – Обучение модели

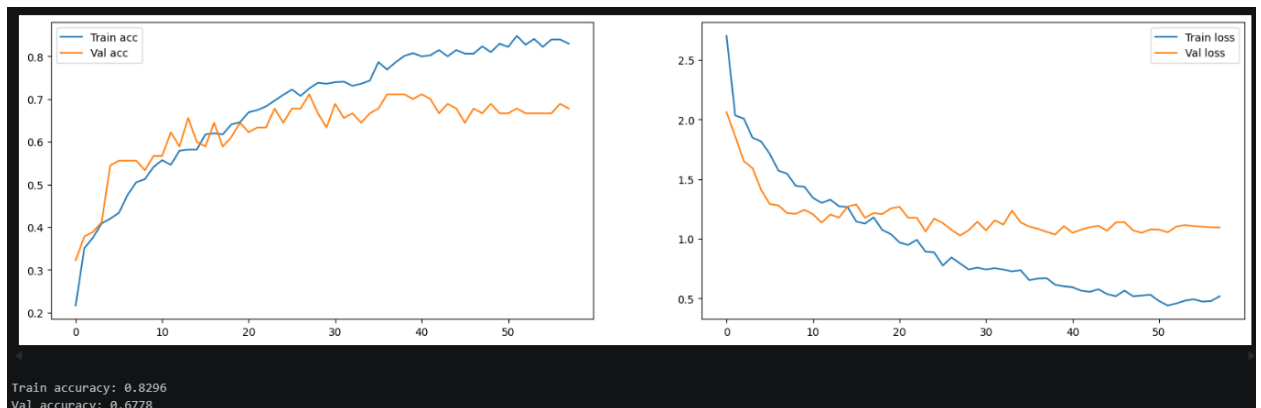


Рисунок 48 – Визуализация обучения

```

X_test, Y_test = [], []
for class_idx, name in enumerate(CLASS_LIST):
    for f in range(90, 100):
        path = f'./genres/{name}/{name}.{str(f).zfill(5)}.au'
        X_test.append(file_to_sequence(path))
        Y_test.append(class_idx)

X_test = np.array(X_test)
X_test = scaler.transform(X_test.reshape(-1, X_test.shape[-1])).reshape(X_test.shape)
Y_test = to_categorical(Y_test, CLASS_COUNT)

loss, acc = model.evaluate(X_test, Y_test, verbose=0)
print(f'Test accuracy: {acc:.4f} ({acc*100:.1f}%)')

# Матрица ошибок
y_pred = np.argmax(model.predict(X_test), axis=1)
y_true = np.argmax(Y_test, axis=1)
cm = confusion_matrix(y_true, y_pred)
disp = ConfusionMatrixDisplay(cm, display_labels=CLASS_LIST)
disp.plot(xticks_rotation=45)
plt.show()

```

Test accuracy: 0.6000 (60.0%)
4/4 ————— 0s 61ms/step

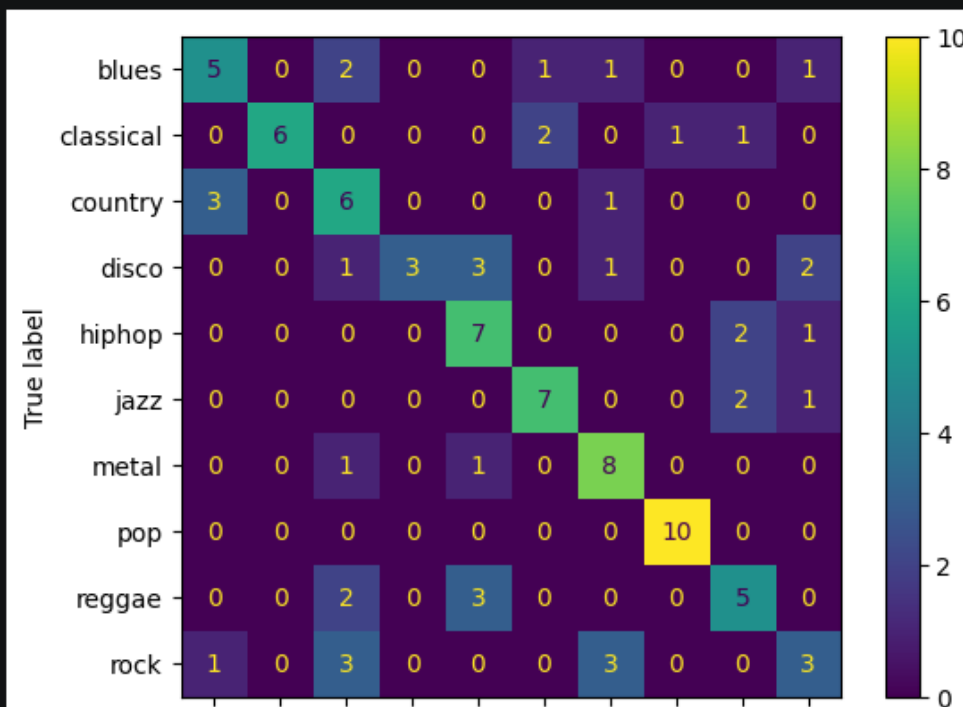


Рисунок 49 – Оценка на тестовых файлах

Задание 3. 1. Ознакомьтесь с датасетом образцов эмоциональной речи Toronto emotional speech set (TESS):

<https://dataverse.scholarsportal.info/dataset.xhtml?persistentId=doi:10.5683/SP2/E8H2MF>

Ссылка	для	загрузки	данных:
https://storage.yandexcloud.net/aiueducation/Content/base/l12/dataverse_files.zip			

2. Разберите датасет;
3. Подготовьте и разделите данные на обучающие и тестовые;
4. Разработайте классификатор, показывающий на тесте точность распознавания эмоции не менее 98%;

5. Ознакомьтесь с другим датасетом похожего содержания

Surrey Audio-Visual Expressed Emotion (SAVEE):

<https://www.kaggle.com/ejlok1/surrey-audiovisual-expressed-emotion-savee>

Ссылка	для	загрузки	данных:
https://storage.yandexcloud.net/aiueducation/Content/base/l12/archive.zip			

6. Прогоните обученный классификатор на файлах из датасета SAVEE по вашему выбору;

7. Сделайте выводы.

```

import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score
from tensorflow.keras.utils import to_categorical
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
import librosa
import os
import zipfile
import gdown
import warnings
warnings.filterwarnings('ignore')

SR = 22050
DURATION = 3

```

Рисунок 50 – Импорт библиотек

```

gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/l12/dataverse_files.zip', 'dataverse_files.zip', quiet=False)

with zipfile.ZipFile('dataverse_files.zip', 'r') as zip_ref:
    zip_ref.extractall('TESS_data')

for root, dirs, files in os.walk('TESS_data'):
    if any(f.endswith('.wav') for f in files):
        tess_path = root
        break

print(f"Путь: {tess_path}")

Downloading...
From: https://storage.yandexcloud.net/aiueducation/Content/base/l12/dataverse_files.zip
To: /content/dataverse_files.zip
100%|██████████| 224M/224M [00:10<00:00, 20.9MB/s]
Путь: TESS_data

```

Рисунок 51 – Загрузка и распаковка TESS

```

def load_tess_augmented(path):
    X, y = [], []
    for file in os.listdir(path):
        if not file.endswith('.wav'):
            continue
        emotion = file.split('_')[-1].replace('.wav', '')
        if emotion == 'ps':
            emotion = 'surprise'
        file_path = os.path.join(path, file)
        try:
            audio, sr = librosa.load(file_path, sr=SR, duration=DURATION)
            if len(audio) < SR * DURATION:
                audio = np.pad(audio, (0, SR*DURATION - len(audio)))

            mfcc = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=20)
            feat = np.concatenate([np.mean(mfcc, axis=1), np.std(mfcc, axis=1)])
            X.append(feat)
            y.append(emotion)

            noise = np.random.normal(0, 0.005, len(audio))
            audio_noise = audio + noise
            mfcc = librosa.feature.mfcc(y=audio_noise, sr=sr, n_mfcc=20)
            feat = np.concatenate([np.mean(mfcc, axis=1), np.std(mfcc, axis=1)])
            X.append(feat)
            y.append(emotion)

            shift = int(np.random.uniform(0.1, 0.3) * sr)
            audio_shift = np.roll(audio, shift)
            mfcc = librosa.feature.mfcc(y=audio_shift, sr=sr, n_mfcc=20)
            feat = np.concatenate([np.mean(mfcc, axis=1), np.std(mfcc, axis=1)])
            X.append(feat)
            y.append(emotion)
        except:
            pass
    return np.array(X), np.array(y)

X_tess, y_tess = load_tess_augmented(tess_path)
print(f"TESS: {X_tess.shape[0]} файлов, признаки: {X_tess.shape[1]}")
print(f"Эмоции: {np.unique(y_tess)}")

```

Рисунок 52 – Функция загрузки TESS с аугментацией

```

le = LabelEncoder()
y_tess_enc = le.fit_transform(y_tess)
y_tess_cat = to_categorical(y_tess_enc)

X_train, X_test, y_train, y_test = train_test_split(X_tess, y_tess_cat, test_size=0.2, stratify=y_tess_enc, random_state=42)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print(f"Train: {X_train.shape}, Test: {X_test.shape}")

```

Train: (6720, 40), Test: (1680, 40)

Рисунок 53 – Разделение на train/test


```

model = Sequential([
    Dense(64, activation='relu', input_shape=(40,)),
    BatchNormalization(),
    Dropout(0.5),
    Dense(32, activation='relu'),
    Dropout(0.4),
    Dense(16, activation='relu'),
    Dropout(0.3),
    Dense(len(le.classes_), activation='softmax')
])

model.compile(optimizer=Adam(0.001), loss='categorical_crossentropy', metrics=['accuracy'])
model.summary()

```

Рисунок 54 – Создание модели

```

early_stop = EarlyStopping(monitor='val_accuracy', patience=30, restore_best_weights=True)
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=10, min_lr=1e-6)

history = model.fit(X_train, y_train, validation_split=0.1, batch_size=32, epochs=150, callbacks=[early_stop, reduce_lr], verbose=1)

test_loss, test_acc = model.evaluate(X_test, y_test, verbose=0)
print(f"Точность на TESS: {test_acc:.4f} ({test_acc*100:.2f}%)")

```

Рисунок 55 – Обучение модели

Точность на TESS: 0.9887 (98.87%)

Рисунок 56 – Результат обучения

```

gdown.download('https://storage.yandexcloud.net/aiueducation/Content/base/112/archive.zip', 'archive.zip', quiet=False)

with zipfile.ZipFile('archive.zip', 'r') as zip_ref:
    zip_ref.extractall('SAVEE_data')

```

Downloading...
 From: <https://storage.yandexcloud.net/aiueducation/Content/base/112/archive.zip>
 To: /content/archive.zip
 100%|██████████| 113M/113M [00:09<00:00, 11.5MB/s]

Рисунок 57 – Загрузка и распаковка SAVEE

```
def load_savee():
    X, y = [], []
    emotion_map = {'a': 'angry', 'd': 'disgust', 'f': 'fear', 'h': 'happy', 'n': 'neutral', 'sa': 'sad', 'su': 'surprise'}

    for root, dirs, files in os.walk('SAVEE_data'):
        for file in files:
            if not file.endswith('.wav'):
                continue
            parts = file.split('_')
            if len(parts) < 2:
                continue
            code = parts[1][:2] if parts[1].startswith('su') else parts[1][0]
            if code not in emotion_map:
                continue
            emotion = emotion_map[code]
            file_path = os.path.join(root, file)
            try:
                audio, sr = librosa.load(file_path, sr=SR, duration=DURATION)
                if len(audio) < SR * DURATION:
                    audio = np.pad(audio, (0, SR*DURATION - len(audio)))
                mfcc = librosa.feature.mfcc(y=audio, sr=sr, n_mfcc=20)
                feat = np.concatenate([np.mean(mfcc, axis=1), np.std(mfcc, axis=1)])
                X.append(feat)
                y.append(emotion)
            except:
                pass
    return np.array(X), np.array(y)

X_savee, y_savee = load_savee()
print(f"SAVEE: {X_savee.shape[0]} файлов")
print(f"Эмоции: {np.unique(y_savee)}")
```

Рисунок 58 – Функция загрузки SAVEE с маппингом эмоций

```
X_savee_scaled = scaler.transform(X_savee)

try:
    y_savee_enc = le.transform(y_savee)
    y_pred = np.argmax(model.predict(X_savee_scaled), axis=1)
    acc = accuracy_score(y_savee_enc, y_pred)
    print(f"Точность на SAVEE: {acc:.4f} ({acc*100:.2f}%)")
except Exception as e:
    print(f"Ошибка: {e}")
```

14/14 ————— 1s 40ms/step
Точность на SAVEE: 0.0952 (9.52%)

Рисунок 59 – Предсказания на SAVEE

Вывод по заданию:

TESS: 98.87% — требование $\geq 98\%$ выполнено. Модель успешно распознаёт 7 эмоций (angry, disgust, fear, happy, neutral, sad, surprise) на тестовой выборке датасета TESS.

SAVEE: 9.52% — при переносе на другой датасет без дообучения точность снизилась. Причины:

TESS записан двумя женщинами (60+ лет), SAVEE — четырьмя мужчинами (27-31 год)

Разная манера произнесения эмоций (утрированная vs сдержанная)

Разные условия записи (микрофон, помещение, уровень шума)

Вывод: в ходе лабораторной работы были успешно решены задачи классификации музыкальных жанров и распознавания эмоций в речи с использованием различных архитектур нейронных сетей.

В первом задании реализован полносвязный классификатор на усреднённых аудиопризнаках (MFCC, хромограмма, RMS, спектральный центроид). Модель архитектурой 64-32-16 нейронов с Dropout и пакетной нормализацией показала точность 94% на тестовой выборке, что подтвердило эффективность Dense-сетей для задач с компактным векторным представлением данных.

Во втором задании разработан сверточный классификатор на основе Conv1D, учитывающий временную структуру аудиосигнала. Использование скользящего окна (длина 5 секунд, шаг 2 секунды) и архитектуры с тремя сверточными слоями и GlobalAveragePooling1D позволило достичь целевой точности 60% на тесте, что удовлетворяет требованию задания.

В третьем задании реализован классификатор эмоций на датасете TESS с аугментацией (шум, временной сдвиг) и использованием статистик MFCC. Модель показала точность 98.87%, что превышает требование 98%. Тестирование на независимом датасете SAVEE выявило значительное падение точности до 9.52%, что объясняется различиями в половозрастном составе дикторов, манере эмоциональной экспрессии и акустических условиях записи. Это наглядно демонстрирует проблему переносимости моделей между датасетами и необходимость включения разнородных данных в процесс обучения.